# Dijkstra's algorithm

# Dijkstra's algorithm

- **Dijkstra's algorithm**: finds shortest (minimum weight) path between a particular pair of vertices in a *weighted* directed graph with nonnegative edge weights (doesn't apply with neg numbers)
  - solves the "one vertex, shortest path" problem
  - basic algorithm concept: create a table of information about the currently known best way to reach each vertex (distance, previous vertex) and improve it until it reaches the best solution

- in a graph where:
  - vertices represent cities,
  - edge weights represent driving distances between pairs of cities connected by a direct road,
    Dijkstra's algorithm can be used to find the shortest route between one city and any other

# Dijkstra pseudocode

worst case: $O(n^2)$

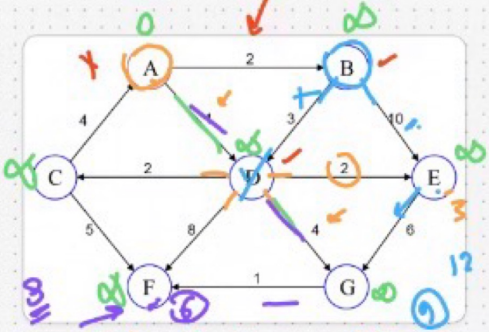
```
Dijkstra(v1, v2):
   for each vertex v:                    // Initialization
      v's distance := infinity.
      v's previous := none.
   v1's distance := 0.
   List := {all vertices}.

   while List is not empty:
      v := remove List vertex with minimum distance.
      mark v as known.
      for each unknown neighbor n of v:
         dist := v's distance + edge (v, n)'s weight.

         if dist is smaller than n's distance:
            n's distance := dist.
            n's previous := v.

   reconstruct path from v2 back to v1,
   following previous pointers.
```

figma.com/board/RtQDjivK9MKz9jym7RuWp0/Untitled?t=E1LGK75z3dTLc8aK-0




Shortest path problem.
from single starting point

1 < ∞ 2 < ∞

| Selected node | B | C | D | E | F | G | |
|---|---|---|---|---|---|---|---|
| | ∞ | ∞ | ∞ | ∞ | ∞ | ∞ | step 1. |
| D | 2 | ∞ | 1 (1st) | ∞ | ∞ | ∞ | ← |
| B | 2 | 3 | 1 (done) | 3 | 9 | 5 | ← |
| C | 2 | 3 | 1 | 3 | 9 | 5 | ← |
| E | 2 | 3 | 1 | 3 | 8 | 5 | ← |
| G | 2 | 3 | 1 | 3 | 8 | 5 | ← |
| F | 2 | 3 | 1 | 3 | 6 | 5 | ← |

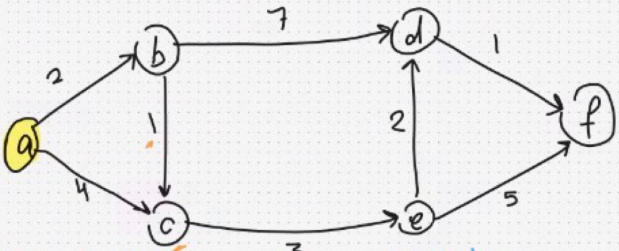
Solution

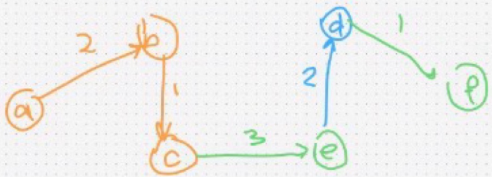
| | |
|---|---|
| A → D | 1 |
| A → B | 2 |
| A → D → C | 3 |
| A → D → E | 3 |
| A → D → G | 5 |
| A - D - G - F | 6 |

The starting point will be always be given

Introduction to Gr | Programming Lan | Parse Trees - YouT | Untitled – FigJam | Course: Design an | Meet - fba-egi
figma.com/board/RtQDjivK9MKz9jym7RuWp0/Untitled?t=E1LGK75z3dTLc8aK-0
Untitled Free
03:00 Share

A→D→G→F 16




Apply Dijkstra algorithm to find shortest path from (a)

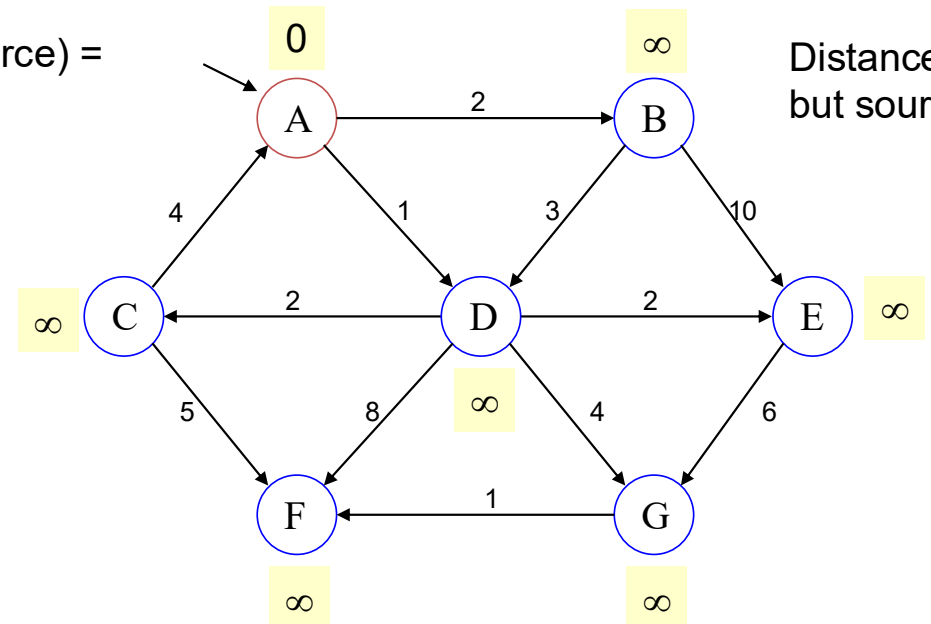
Solution

| | |
|---|---|
| a→b | 2 |
| a→b→c | 3 |
| a→b→c→e | 6 |
| a→b→c→e→d | 8 |
| a→b→c→e→d→f | 9 |

| selected node | b | c | d | e | f |
|---|---|---|---|---|---|
| | ∞ | ∞ | ∞ | ∞ | ∞ |
| b | 2 | 4 | ∞ | ∞ | ∞ |
| c | X | 3 | 9 | 6 | ∞ |
| e | X | X | 9 | X | 11 |
| d | X | X | 8 | X | 9 |
| f | X | X | Y | X | 9 |

# Example: Initialization

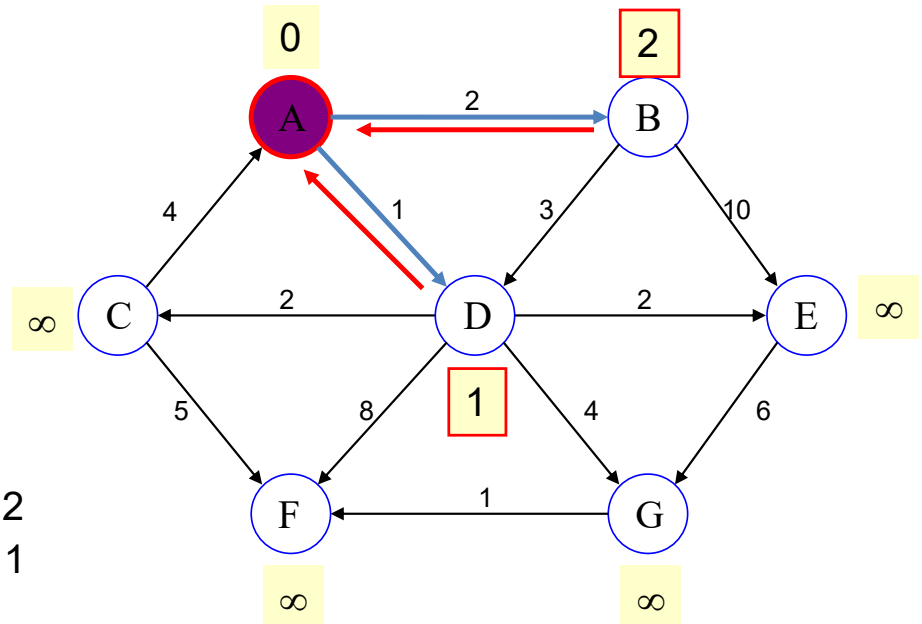
Distance(source) = 0




Distance (all vertices but source) = ∞

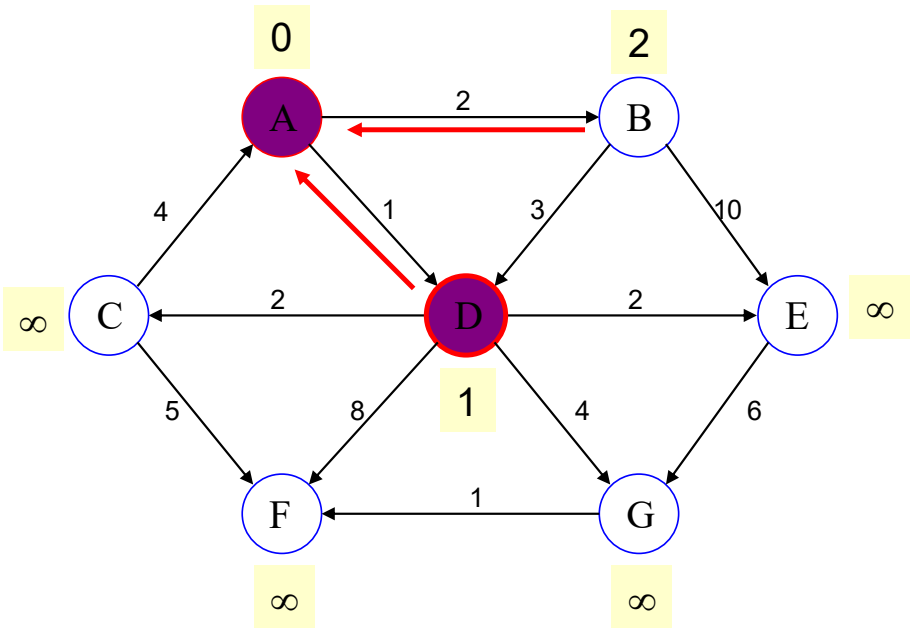
Pick vertex in List with minimum distance.

# Example: Update neighbors' distance



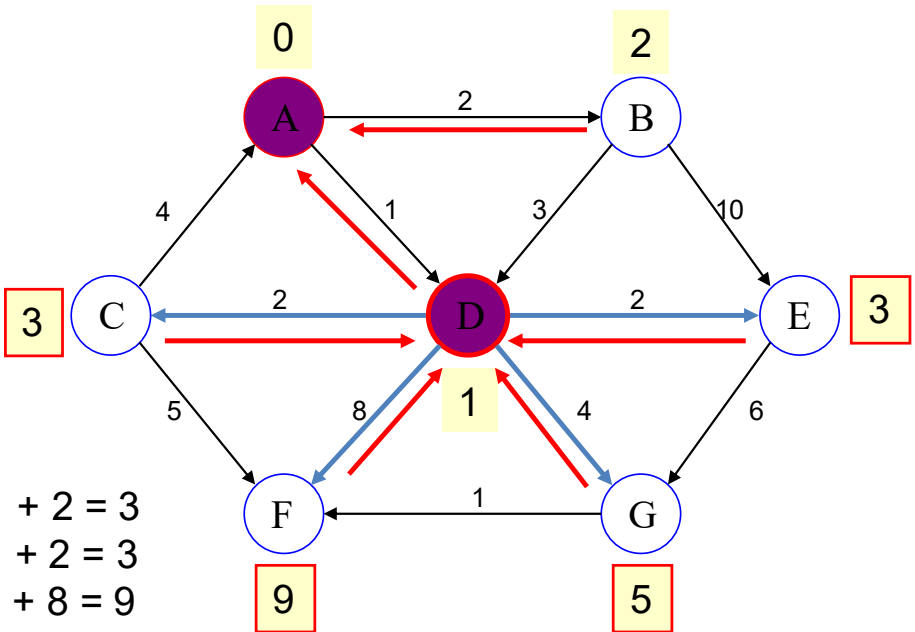

Distance(B) = 2
Distance(D) = 1

# Example: Remove vertex with minimum distance




Pick vertex in List with minimum distance, i.e., D

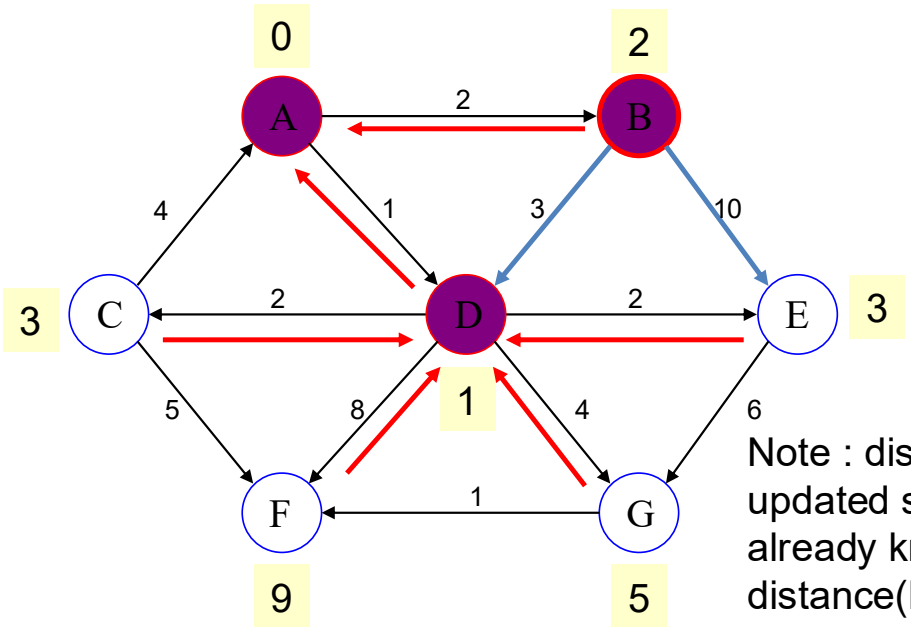
# Example: Update neighbors




Distance(C) = 1 + 2 = 3
Distance(E) = 1 + 2 = 3
Distance(F) = 1 + 8 = 9
Distance(G) = 1 + 4 = 5

# Example: Continued...

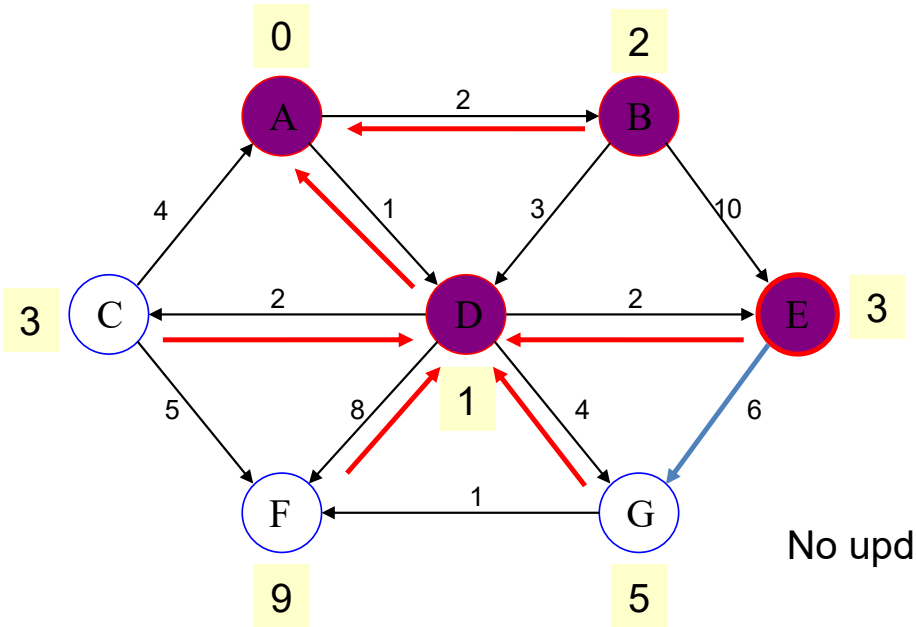
Pick vertex in List with minimum distance (B) and update neighbors




Note : distance(D) not updated since D is already known and distance(E) not updated since it is larger than previously computed

# Example: Continued...

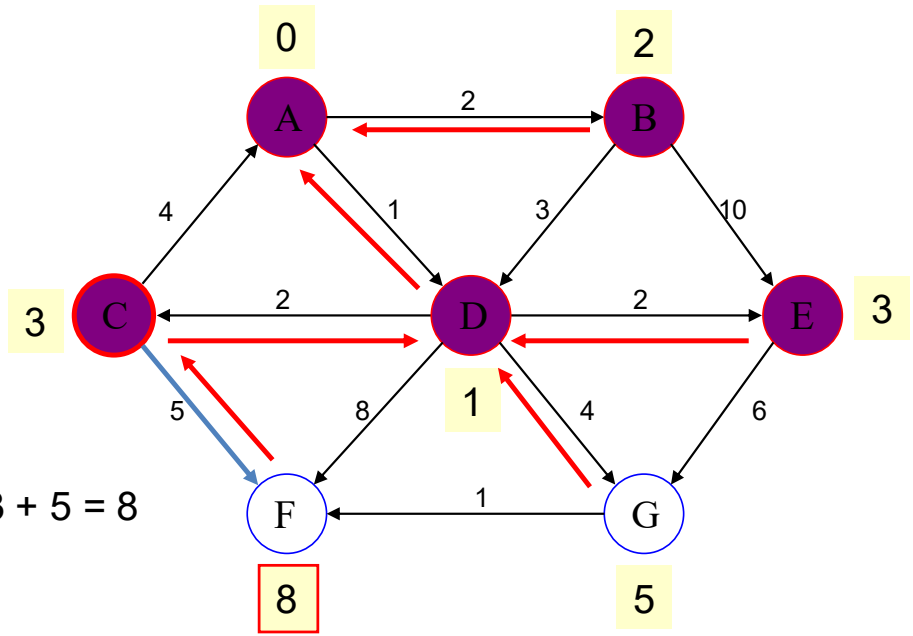
Pick vertex List with minimum distance (E) and update neighbors




No updating

# Example: Continued...

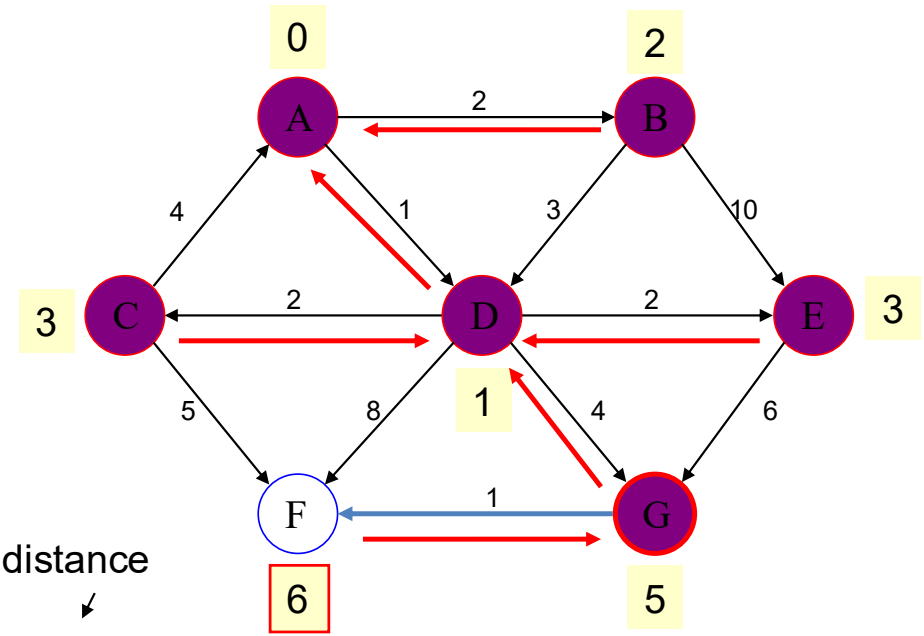
Pick vertex List with minimum distance (C) and update neighbors




Distance(F) = 3 + 5 = 8

# Example: Continued...

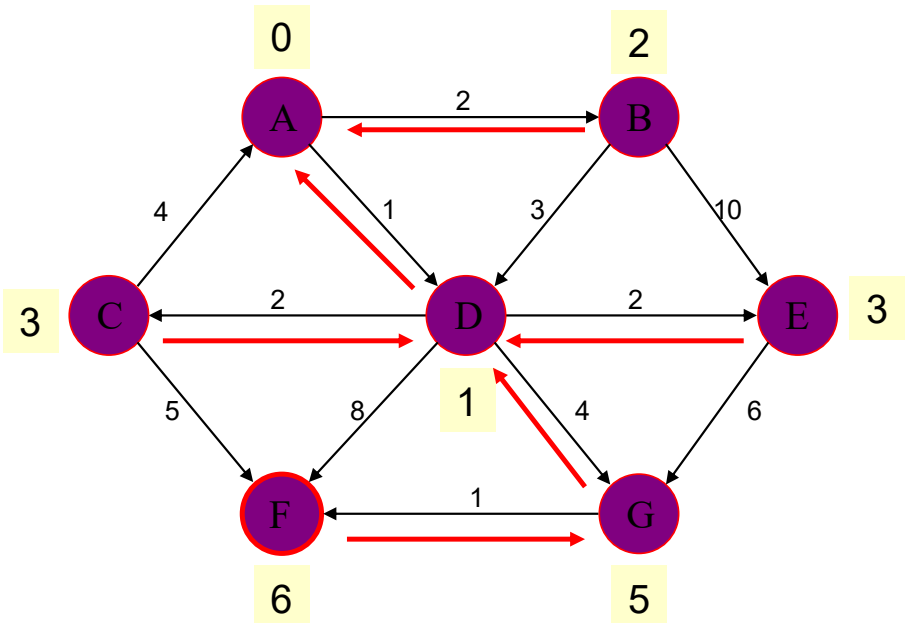
Pick vertex List with minimum distance (G) and update neighbors




Previous distance

Distance(F) = min (8, 5+1) = 6

# Example (end)




Pick vertex not in S with lowest cost (F) and update neighbors

# Time Complexity: Using List

The simplest implementation of the Dijkstra's algorithm stores vertices in an ordinary linked list or array

- Good for dense graphs (many edges)

- $|V|$ vertices and $|E|$ edges
- Initialization $O(|V|)$
- While loop $O(|V|)$
  - Find and remove min distance vertices $O(|V|)$
  - Potentially $|E|$ updates
    - Update costs $O(1)$
- Reconstruct path $O(|E|)$

Total time $O(|V^2| + |E|) = O(|V^2|)$

# Time Complexity: Priority Queue

For sparse graphs, (i.e. graphs with much less than $|V^2|$ edges)
Dijkstra's implemented more efficiently by *priority queue*

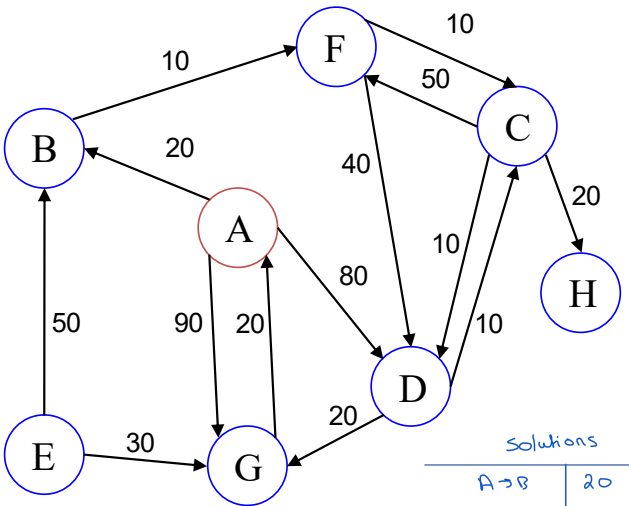
- Initialization O(|V|) using O(|V|) buildHeap
- While loop O(|V|)
  - Find and remove min distance vertices O(log |V|) using O(log |V|) deleteMin
  - Potentially |E| updates
    - Update costs O(log |V|) using decreaseKey
- Reconstruct path O(|E|)

Total time O(|V|log|V| + |E|log|V|) = O(|E|log|V|)

- |V| = O(|E|) assuming a connected graph

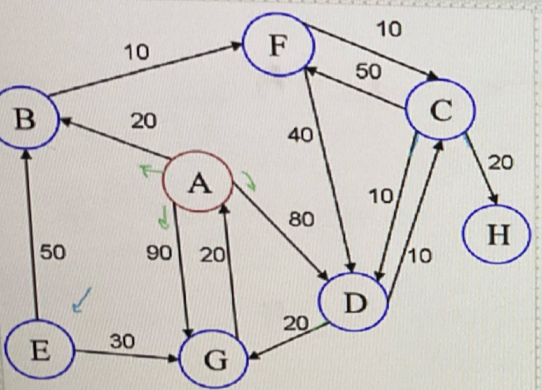
# Dijkstra's Exercise

- Use Dijkstra's algorithm to determine the lowest cost path from vertex A to all of the other vertices in the graph. Keep track of previous vertices so that you can reconstruct the path later.

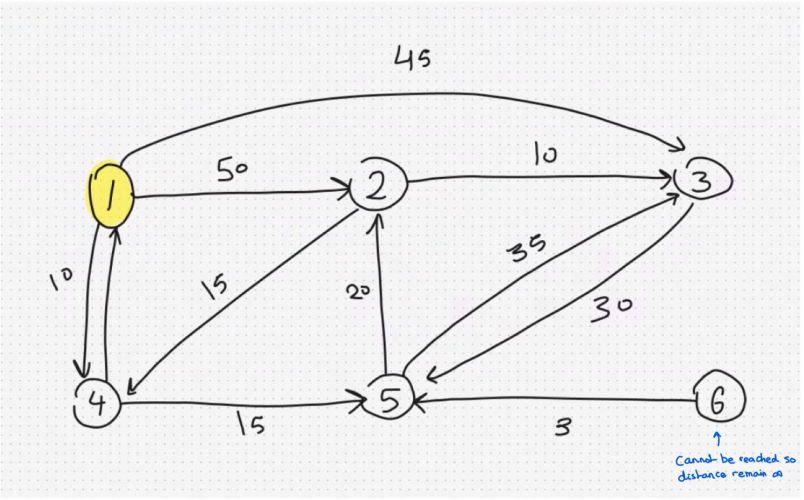



| Selected node | B | C | D | E | F | G | H |
|---|---|---|---|---|---|---|---|
| | ∞ | ∞ | ∞ | ∞ | ∞ | ∞ | ∞ |
| B | 20 | ∞ | 80 | ∞ | ∞ | 90 | ∞ |
| F | x | ∞ | 80 | ∞ | 30 | 90 | ∞ |
| C | x | 40 | 70 | ∞ | x | 90 | ∞ |
| D | x | x | 50 | ∞ | x | 90 | 60 |
| H | x | x | x | ∞ | x | 70 | 60 |
| G | x | x | x | ∞ | x | 70 | x |
| E | x | x | x | ∞ | x | x | x |

| Solutions | |
|---|---|
| A→B | 20 |
| A→B→F | 30 |
| A→B→F→C | 40 |
| A→B→F→C→D | 50 |
| A→B→F→C→H | 60 |
| A→B→F→C→D→G | 70 |




| Selected | B | C | D | E | F | G | H |
|---|---|---|---|---|---|---|---|
| | ∞ | ∞ | ∞ | ∞ | ∞ | ∞ | ∞ |
| B | 20 ✓ | ∞ | 80 | ∞ | ∞ | 90 | ∞ |
| F | X | ∞ | 80 | ∞ | 30 ✓ | 90 | ∞ |
| C | X | 40 | 70 | ∞ | X | 90 | ∞ |
| D | X | X | 50 | ∞ | X | 90 | 60 |
| H | X | X | X | ∞ | X | 70 | 60 |
| G | X | X | X | ∞ | X | 70 | X |
| E | X | X | X | ∞ | X | X | X |

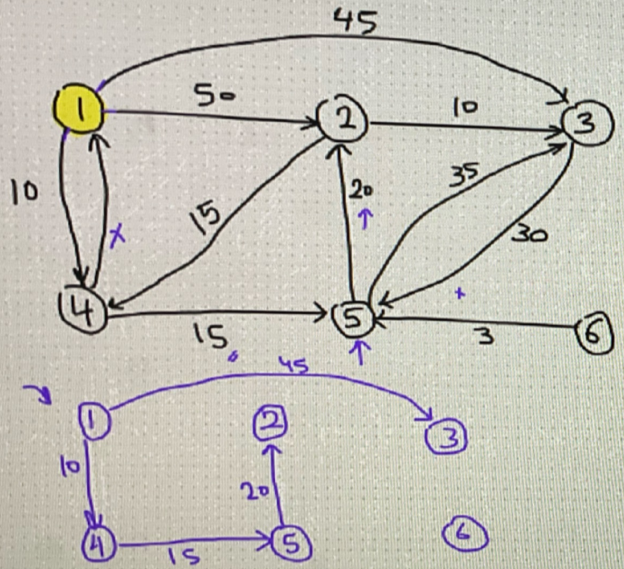



| Selected node | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|
| | ∞ | ∞ | ∞ | ∞ | ∞ |
| 4 | 50 | 45 | 10 | ∞ | ∞ |
| 5 | 50 | 45 | 10 | 25 | ∞ |
| 2 | 45 | 45 | 10 | 25 | ∞ |
| 3 | 45 | 45 | 10 | 25 | ∞ |
| 6 | 45 | 45 | 10 | 25 | ∞ |

| Solutions | |
|---|---|
| 1 → 4 | 10 |
| 1 → 4 → 5 | 25 |
| 1 → 4 → 5 → 2 | 45 |
| 1 → 3 | 45 |

duHub
Untitled – FigJam
Drafts – Figma
figma.com/board/RF9YxGNWS2emUkneqwlrPp/Untitled?node-id=0-1&p=f&t=chw4uXLsVKSxxPvJ-0
Free

# Shortest path







| selected | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|
| | ∞ | ∞ | ∞ | ∞ | ∞ |
| 4 | 50 | 45 | (10) | ∞ | ∞ |
| ✓ 5 | 50 | 45 | X | (25) | ∞ |
| 2 | (45) | 45 | X | X | ∞ |
| 3 | X | (45) | X | X | ∞ |
| 6 | X | X | X | X | ∞ |

Search
ENG